

Wykład 6

Układy czasowo-licznikowe Liczniki Timery/Tajmery

Układy czasowo licznikowe

Charakterystyka ogólna:

Układy czasowo-licznikowe są licznikami cyfrowymi wyposażonymi w zaawansowane tryby wyzwalania i programowanie źródła sygnału zegarowego

Układy licznikowe w mikrokontrolerach służą do:

- zliczania zdarzeń
- odmierzania czasu
- generacji przebiegów o programowanej częstotliwości i wypełnieniu

Układy czasowo licznikowe

W architekturze AVR ATmega dostępne są co najmniej dwa układy czasowo-licznikowe

W ATmega implementowane są liczniki 8- i 16-bitowe

Mikrokontroler ATmega32 posiada:

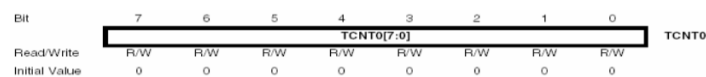
- 2 liczniki 8-bitowe timer/counter0 i timer/counter2
- 1 licznik 16-bitowy timer/counter1

Układy czasowo licznikowe

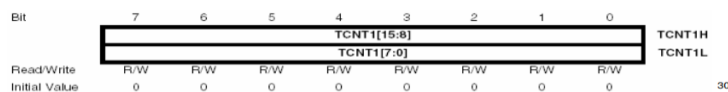
Rejestry funkcyjne:

- Rejestr **TCNTn** (n jest numerem licznika) zawiera aktualną wartość zliczoną przez licznik i jest rejestrem 8-bitowym lub parą rejestrów **TCNTnH** i **TCNTnL** dla licznika 16-bitowego.
- W przypadku licznika 16-bitowego **TCNTnH** jest rejestrem buforowym i wymagana jest ustalona sekwencja zapisu w celu prawidłowej inicjalizacji licznika
- Przepiętnienie licznika wywołuje zgłoszenie przerwania

8-bitowy rejestr **TCNT0**:



Para rejestrów **TCNT1H** i **TCNT1L**:



30

Układy czasowo licznikowe

Rejestry funkcyjne:

- Aktualna wartość w liczniku jest porównywana z zawartością rejestru **OCRn** lub **OCRnH** i **OCRnL**
- Z licznikiem 1. w ATmega32 są skojarzone 2 pary rejestrów: **OCR1AH(L)** i **OCR1BH(L)**
- W wyniku zrównania się wartości w liczniku i rejestrze OCR jest zgłaszane przerwanie
- Wynik operacji porównania może służyć do generacji sygnału o programowanym wypełnieniu

Układy czasowo licznikowe

Rejestry funkcyjne:

- Do sterowania trybem pracy licznika służy rejestr **TCCRn**.
- Bity **CSn** określają źródło sygnału taktującego licznik
- Bity **WGMn** określają tryb pracy licznika
- Znaczniki **COMn** wybierają sposób generacji sygnałów wyjściowych
- Przykład - rejestr sterujący pracą licznika 0 (TCCR0)

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Znaczenie bitu **FOC0** i bitów **COM0x** zależy od trybu pracy.

Układy czasowo licznikowe

Rejestry funkcyjne:

- Znaczenie bitów **WGM0x** - tryb pracy:

Mode	WGM01 (CTC0)	WGM00 (PWM0)	Timer/Counter Mode of Operation	TOP	Update of OCR0	TOV0 Flag Set-on
0	0	0	Normal	0xFF	Immediate	MAX
1	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	1	0	CTC	OCR0	Immediate	MAX
3	1	1	Fast PWM	0xFF	BOTTOM	MAX

- Znaczenie bitów **CS0x** – źródło sygnału taktującego:

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{IO}/(No\ prescaling)$
0	1	0	$clk_{IO}/8$ (From prescaler)
0	1	1	$clk_{IO}/64$ (From prescaler)
1	0	0	$clk_{IO}/256$ (From prescaler)
1	0	1	$clk_{IO}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

Układy czasowo licznikowe

Rejestry funkcyjne:

- Zadaniem preskalera jest dostarczenie sygnału taktującego o programowanej w szerokim zakresie częstotliwości (podział 1:1024)
- Tryb pracy normalnej – zliczanie do wartości maksymalnej
- Porównywanie – porównanie zawartości licznika i rejestru OCRn wywołuje różne, zaprogramowane akcje
- Tryb CTC – Clear Timer on Compare - zerowanie licznika po wykryciu zgodności porównania
- Tryby PWM – generacja sygnału prostokątnego o programowanej częstotliwości i wypełnieniu

Układy czasowo licznikowe

Załączanie przerwań w rejestrze TIMSK

- Bity **TOIE_n** zezwalają na przerwanie od przepełnienia timera **n**
- Bity **OCIE_{nX}** zezwalają na przerwanie od odpowiedniego układu porównania timera **n**

Bit	7	6	5	4	3	2	1	0	
	OCIE2	TOIE2	TICIE1	OCIE1A	OCIE1B	TOIE1	OCIE0	TOIE0	TIMSK
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

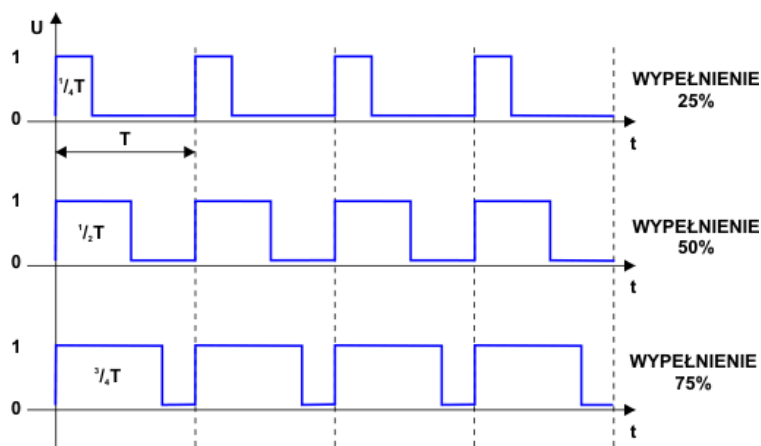
Znaczniki przerwań w rejestrze TIFR

- Bity **TOV_n** zgłaszają przerwanie od przepełnienia timera **n**
- Bity **OCF_{nX}** zgłaszają przerwanie od odpowiedniego układu porównania timera **n**

Bit	7	6	5	4	3	2	1	0	
	OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0	TIFR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Układy czasowo licznikowe – tryb PWM

Modulacja Szerokości Impulsów (ang. Pulse Width Modulation – PWM)

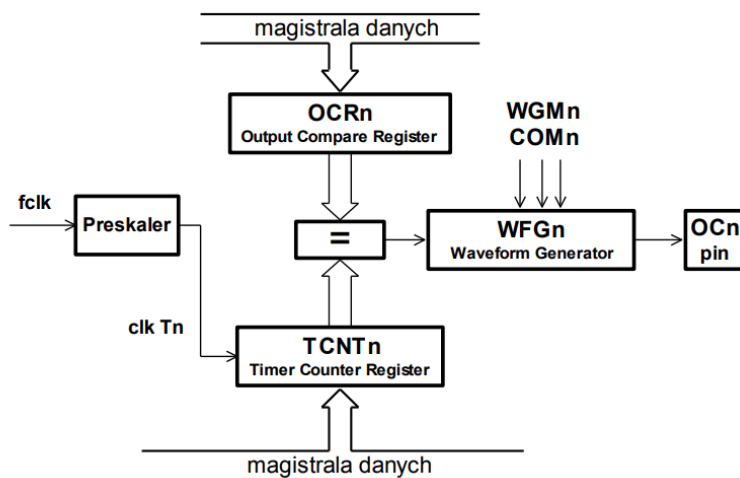


Zastosowanie:

- Dostarczanie zmiennej mocy do układów takich jak LED, silniki DC lub sterowanie serwomechanizmami

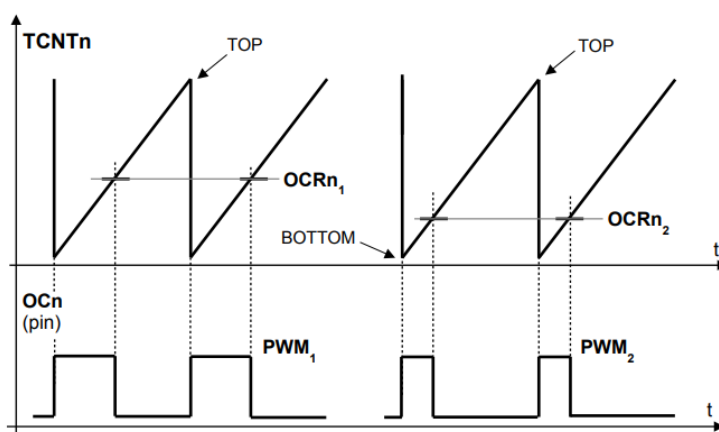
Układy czasowo licznikowe – tryb PWM

Uproszczony schemat blokowy generatora PWM



Układy czasowo licznikowe – tryb PWM

Zależność współczynnika wypełnienia PWM od zawartości rejestru OCRn w trybie Fast PWM



TOP – maksymalna zawartość rejestru TCNTn

BOTTOM – minimalna zawartość rejestru TCNTn

Układy czasowo licznikowe – tryb PWM

Generatory PWM mogą pracować w różnych trybach, które wybierane są ustawieniami rejestrów konfiguracyjnych TCCRn

TCCRn

FOC	WGMn0	COMn1	COMn0	WGMn1	CSn2	CSn1	CSn0
-----	-------	-------	-------	-------	------	------	------

FOC - wymuszanie stanu wyjścia OCn

WGMn1, WGMn0 - wybór trybu pracy generatora PWM.

COMn1, COMn0 - wybór trybu pracy wyjścia OCn.

CSn2 – CSn0 - ustawienie podziału preskalera.

WGMn1	WGMn0	Opis trybu pracy	Wartość TOP
0	0	Normalny (Normal)	0xFF
0	1	Symetryczny PWM (PWM, Phase Correct)	0xFF
1	0	CTC	OCRn
1	1	Szybki PWM (Fast PWM)	0xFF

Układy czasowo licznikowe – tryb PWM

Tryby pracy wyjścia OCn dla trybu Fast PWM

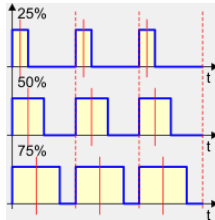
COMn1	COMn0	Opis trybu pracy
0	0	Pin portu
0	1	Niewykorzystany
1	0	Zeruj stan OCn przy zgodności OCRn i TCNTn ustaw stan OCn przy przepełnieniu TCNTn (BOTTOM) PWM prosty
1	1	Ustaw stan OCn przy zgodności OCRn i TCNTn zeruj stan OCn przy przepełnieniu TCNTn (BOTTOM) PWM zanegowany

Wyboru odpowiedniej wartości okresu przebiegu PWM dokonuje się bitami CSn2 – CSn0 ustalającymi stopień podziału preskalera danego timera.

CS02	CS01	CS00	Współczynnik podziału preskalera
0	0	0	Brak taktowania, timer zatrzymany
0	0	1	fclk
0	1	0	fclk/8
0	1	1	fclk/64
1	0	0	fclk/256
1	0	1	fclk/1024
1	1	0	Licznik, zegar T0, zbocze opadające
1	1	1	Licznik, zegar T0, zbocze narastające

Układy czasowo licznikowe – tryb PWM

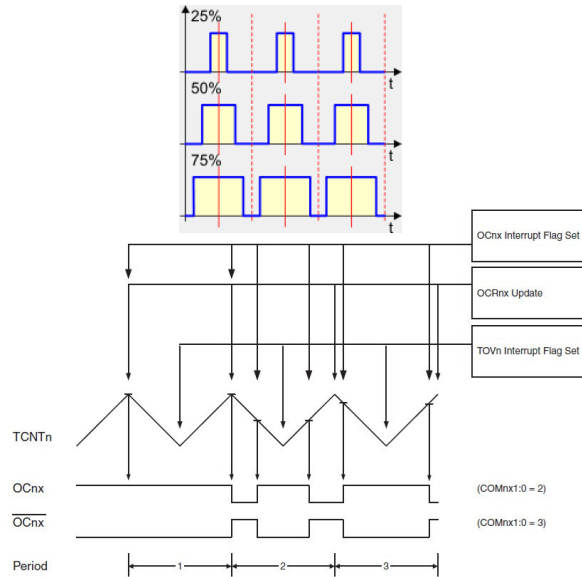
Tryb Fast PWM



W szybkim trybie PWM zmiana wypełnienia powoduje również przesunięcie fazy sygnału

Potencjalna wada PC PWM:
maksymalna częstotliwość 2
razy mniejsza

Tryb PWM z korekcją fazową (Phase Correct PWM Mode)



Układy czasowo licznikowe – tryb PWM

Przykład konfiguracji sprzętowego FAST PWM

```

DDR0 |= (1<<PB3); // ustawienie końcówki OC0 (PB3) sprzętowy PWM jako WYJŚCIE
TCCR0 |= (1<<WGM01)|(1<<WGM00); // tryb Fast PWM
TCCR0 |= (1<<COM01); // clear at TOP
TCCR0 |= (1<<CS00); // prescaler = 1
  
```

Odpowiednie wypełnienie sygnału uzyskuje się poprzez ustawienie wartości bitu

```
OCR0 = (0-255);
```

Potencjalna wada sprzętowego PWM: częstotliwości sygnału ściśle określone:

$$f_{OCnXPWM} = \frac{f_{clk} \cdot I/O}{N \cdot 256}$$

Np.:

$f_{clk}=16 \text{ MHz}$, $prescaler=1$, $f_{PWM}=62,5 \text{ kHz}$

$f_{clk}=16 \text{ MHz}$, $prescaler=256$, $f_{PWM}=976,56 \text{ Hz}$

$f_{clk}=16 \text{ MHz}$, $prescaler=1024$, $f_{PWM}=61 \text{ Hz}$

Układy czasowo licznikowe – tryb PWM

Programowy PWM

W Atmega32 są „tylko” trzy układy licznikowe i może być ich za mało do obsługi kilku urządzeń, ale jest możliwość wykonania programowego PWM. Mechanizm oparty na przerwaniach

```
// ustawienie pinów kanałów programowych PWM jako WYJŚCIA
DDRC |= (1<<PC0)|(1<<PC1)|(1<<PC2);
// wyłączenie diod LED podłączonych katodami do wyjść
PORTC |= (1<<PC0)|(1<<PC1)|(1<<PC2);
// ustawienia TIMER2 w tryb CTC
TCCR2 |= (1<<WGM21); // tryb CTC
TCCR2 |= (1<<CS20); // prescaler = 1
OCR2 = 199; // dodatkowy podział częstotliwości przez 200
TIMSK |= (1<<OCIE2); // zezwolenie na przerwanie CompareMatch
sei(); // odblokowanie globalne przerw

...

// ciało procedury obsługi przerwania Compare Match Timera2
ISR( TIMER2_COMP_vect )
{
    static uint8_t cnt; // definicja naszego licznika PWM
    // bezpośrednie sterowanie wyjściami kanałów PWM
    if(cnt>=pwm1) PORTC |= (1<<PC0); else PORTC &= ~(1<<PC0);
    if(cnt>=pwm2) PORTC |= (1<<PC1); else PORTC &= ~(1<<PC1);
    if(cnt>=pwm3) PORTC |= (1<<PC2); else PORTC &= ~(1<<PC2);
    cnt++; // zwiększanie licznika o 1
}
```

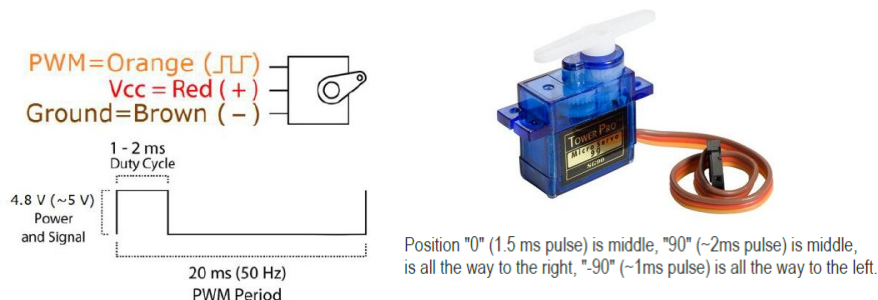
Układy czasowo-licznikowe cd. Przerwania

Układy czasowo licznikowe – tryb PWM

Serwomechanizm

- to silnik, przekładnia oraz dedykowany sterownik zamknięty w jednej obudowie.
- Napędy te nie są przystosowane do wykonywania pełnego obrotu.
- Najczęściej serwomechanizmy mogą poruszać zamontowanym ramieniem o kąt 0-180°
- znają one swoją aktualną pozycję, więc nie występują narastające błędy pozycji.

Serwo SG90



Sprzętowy PWM nie jest idealny do sterowania serwow, do tego często potrzeba sterować wieloma urządzeniami na raz

Układy czasowo licznikowe

Sterowanie serwow SG90

- Zakładamy, że na pełen zakres obrotu (0-180°) serwa będzie przypadało 100 pozycji
- Przy 1 ms wypełnienia sterującej pełnym zakresem, krok sterujący wynoszący 10 μs pozwalają przesunąć serwo o 1/100 pełnego zakresu obrotu
- Na pełen okres sygnału PWM wypada $20\text{ms}/10\mu\text{s} = 2000$ kroków sterujących

Sposób rozwiązania:

Generowanie przerwanie co 10 μs, zliczanie impulsów celu osiągnięcia odpowiedniego wypełnienia i okresu.

Wyznaczenie parametrów licznika:

- przy częstotliwości taktowania 16 MHz, i preskalerze =8, okres sygnału Timera wynosi 0,5 μs, czyli 1 krok sterujący wynosi 20 taktów timera.
- Ponieważ do zliczamy 2000 kroków sterujących trzeba wykorzystać licznik 16-bitowy

Układy czasowo licznikowe

Sterowanie serwem SG90

```
// ustawienie pinów kanałów programowych PWM jako WYJŚCIA
DDRC |= (1<<PC0)|(1<<PC1)|(1<<PC2);
// ustawienie poziomu 0 na wyjściach
PORTC &= ~( (1<<PC0)|(1<<PC1)|(1<<PC2) );

//ustawienie timera 1
TCCR1B |= (1<<WGM12); //tryb ctc
TCCR1B |= (1<<CS11); // preskaler 8
OCR1A = 19; // zliczanie 20 taktów zegara czyli 10 us
TIMSK |= (1<<OCIE1B); // zezwolenie na przerwanie CompareMatch
sei(); // odblokowanie globalne przerw

....

ISR(TIMER1_COMPB_vect)
{
    // bezpośrednie sterowanie wyjściami kanałów PWM
    if(cnt<=pwm1) PORTC |= (1<<PC0); else PORTC &= ~(1<<PC0);
    if(cnt<=pwm2) PORTC |= (1<<PC1); else PORTC &= ~(1<<PC1);
    if(cnt<=pwm3) PORTC |= (1<<PC2); else PORTC &= ~(1<<PC2);
    if(cnt<=pwm3) PORTD = 0xFF; else PORTD = 0;
    cnt++; // zwiększanie licznika o 1
    if (cnt==1999) cnt=0;
}
```

System przerw

- Układ mikrokontrolera musi działać dynamicznie – odpowiednie podprogramy muszą być wykonywane selektywnie i w kolejności zależnej od stanu układów wewnętrznych ale i od sygnałów docierających z zewnątrz
- Jeśli jakiś układ zgłasza potrzebę jego obsługi (tzw. żądanie przerwania), mikrokontroler, jeśli program to przewiduje, przerywa bieżącą pracę i przystępuje do wykonywania podprogramu przyporządkowanego danemu źródłu przerwania. Po zakończeniu podprogramu układ wraca do poprzednie wykonywanych czynności.
- Należy uaktywnić obsługę przerw (po inicjalizacji jest to wyłączone)
- Należy też w rejestrze funkcyjnym danego urządzenia ustawić odpowiednio bit sterujący lub zestaw bitów w pozwalający bardziej szczegółowo sterować przerwami w danym module

System przerwań

Priorytety i wektory przerwań

- W przypadku zgłoszenia więcej niż jednego żądania przerwania w jednym cyklu zegarowym, kolejność ich obsługi zależy od ich *priorytetu*. Priorytety są ustalane odgórnie, dla danego typu układu.
- *Wektor przerwań* - to adres w pamięci programu pod którym powinien znajdować się podprogram obsługi tego przerwania.
- Pojemność pamięci przydzielonej danemu wektorowi to 1 lub 2 słowa rozkazowe, więc w praktyce może się tam znaleźć instrukcja skoku do miejsca gdzie znajduje się właściwy podprogram.
- W przypadku gdy zostanie włączona globalna obsługa przerwań, po czasie gdy pojawiły się już jakieś przerwania, obsługiwane w pierwszej kolejności będą te które mają wyższy priorytet a nie te które wystąpiły wcześniej,

System przerwań

Rozłożenie wektorów przerwań w pamięci programu (ATmega32)

wektor	Adres w pamięci programu	źródło	Rodzaj przerwania
1	\$000	RESET	Sygnał z zewnątrz, reset po włączeniu, reset Brown-out, reset Watch-dog, reset JTAG AVR
2	\$002	INT0	Zewnętrzne przerwanie 0
3	\$004	INT1	Zewnętrzne przerwanie 1
4	\$006	INT2	Zewnętrzne przerwanie 2
5	\$008	TIMER2 COMP	Porównanie w TIMER 2
6	\$00A	TIMER2 OVF	Przepelnienie w TIMER 2
7	\$00C	TIMER1 CAPT	Przechwycenie w TIMER 1
8	\$00E	TIMER1 COMPA	Porównanie A w TIMER 1
9	\$010	TIMER1 COMPB	Porównanie B w TIMER 1
10	\$012	TIMER1 OVF	Przepelnienie TIMER 1
11	\$014	TIMER0 COMP	Porównanie TIMER 0
12	\$016	TIMER0 OVF	Przepelnienie w TIMER 0
13	\$018	SPI, STC	Zakończenie transmisji przez SPI
14	\$01A	USART, RXC	Zakończenie odbierania przez USART
15	\$01C	USART, UDRE	Pusty rejestr danych USART
16	\$01E	USART, TXC	Zakończenie wysyłania przez USART
17	\$020	ADC	Zakończenie przetwarzania przez ADC
18	\$022	EE_RDY	Stan gotowości EEPROM
19	\$024	ANA_COMP	Komparator analogowy
20	\$026	TWI	Interfejs I2C
21	\$028	SPM_RDY	Gotowość na umieszczenie programu

Przerwane zewnętrzne może być wyzwolone zboczem narastającym, opadającym, lub niskim poziomem (INT2 tylko zboczem).

System przerwań

Znaczniki przerwań

- Prawie każdemu przerwaniu przyporządkowany jest dedykowany rejestr w przestrzeni adresowej wejścia-wyjścia w którym znajduje się znacznik przerwania (*interrupt flag*)
- Aktywność tego bitu oznacza wystąpienie żądania i ustawiany jest przez odpowiednie źródło żądania
- Zerowanie następuje przeważnie automatycznie w chwili gdy mikrokontroler przechodzi do obsługi przerwania
- Nawet jeśli obsługa przerwań nie jest uaktywniona, znaczniki przerwania działają i możliwe jest ich wykorzystanie w programie do sterowania jego działaniem (mechanizm *polling* - odpytywania)
- Wtedy wymagane jest „ręcznie” zerowanie znacznika

System przerwań

Znaczniki przerwań

- jeśli obsługa przerwań jest uaktywniona, i znacznik przerwania został ustawiony, ale przerwanie nie może być natychmiast obsłużone (np., obsługiwane jest inne przerwanie) można programowo je wyzerować (i przerwanie nie zostanie zauważone)
- Istnieją przerwania „jednorazowe” z mechanizmem znacznikowym oraz „wielokrotne” bez tego mechanizmu i podprogram obsługi jest realizowany tak długo jak przerwanie jest aktywne

System przerwań

Obsługa przerwań

1. Automatyczne wyłączenie globalnego zezwolenie na przerwania ($I=0$) i zachowanie na stosie licznika programu (4 cykle)
2. Wykonanie instrukcji umieszczonej w wektorze zgłoszonego przerwania (przeważnie skok) (2-3 takty zegara)
3. Wykonanie podprogramu; zakończenie instrukcją powrotu *reti* (włączenie globalnego zezwolenia na przerwania $I=1$) lub *ret* (nie powoduje odblokowania przerwań $I=0$)
4. Zdjęcie ze stosu adresu powrotnego i wykonanie skoku
5. Jeśli w trakcie wykonywania podprogramu przerwania pojawi się kolejne przerwanie, to wykonanie zostanie zawieszone do momentu ponownego ustawienia $I=1$ (automatycznie lub manualnie)
6. Po zakończeniu obsługi jednego przerwania i rozpoczęciem obsługi kolejnego mikrokontroler wykonuje co najmniej jedną instrukcję programu głównego

Podprogram powinien być możliwie jak najkrótszy (zaprzestanie działania głównego programu, blokowanie innych przerwań, możliwość przepełnienia stosu)